Gestion des Processus et Signaux POSIX sous Unix/Linux

Thème principal : Architecture et Systèmes d'Exploitation : Cycle de vie des processus, ordonnancement multitâche et communication par signaux POSIX

Synthèse exécutive : Ce cours aborde le fonctionnement fondamental des systèmes d'exploitation Unix/Linux concernant l'exécution des tâches. Il détaille la transition entre un programme statique (fichier sur disque) et un processus dynamique (instance active en mémoire RAM). Pour orchestrer l'exécution simultanée de multiples applications sur les cœurs du processeur, le noyau s'appuie sur le multitâche préemptif et un modèle d'états à trois niveaux (Élu, Prêt, Bloqué). Le cours explique la filiation arborescente des processus (PID/PPID) issue du processus racine systemd/init (PID 1), l'utilisation des appels système fork() et exec(), ainsi que la communication asynchrone et le contrôle des tâches via les signaux POSIX (SIGINT, SIGTERM, SIGKILL, SIGSTOP, SIGCONT).

1. Du Programme Statique au Processus Dynamique : Cycle de Vie, Attributs et Multitâche

Pour exécuter un code informatique, le système d'exploitation doit faire basculer le programme (données passives sur disque) vers l'état de processus (entité active en mémoire RAM). L'ordonnanceur du noyau orchestre cette activité via un modèle à trois états fondamentaux. Lorsqu'un processus est à l'état 'Élu', il occupe physiquement un cœur de processeur et ses instructions sont exécutées. Si son quantum de temps CPU expire, le noyau le bascule à l'état 'Prêt' et attribue le CPU à un autre processus. Si le processus déclenche une demande de lecture sur disque ou attend une saisie clavier, il bascule à l'état 'Bloqué' afin de ne pas gaspiller inutilement des cycles CPU. Dès que la ressource demandée devient disponible, le noyau bascule le processus à l'état 'Prêt' (et non directement à l'état 'Élu'), le replaçant dans la file d'attente de l'ordonnanceur. Chaque processus possède un identifiant unique (PID - Process ID) et référence le PID de son créateur (PPID - Parent Process ID). Lors de la création d'une sous-tâche dans un terminal, le processus shell parent effectue l'appel système fork() qui duplique la structure du parent. Ensuite, le processus fils appelle exec() pour charger le nouveau binaire en mémoire.

- • Programme vs Processus : Un programme est un fichier exécutable passif résidant sur disque. Un processus est son instance dynamique chargée en mémoire RAM avec des ressources allouées.
- • Multitâche Préemptif : L'ordonnanceur (scheduler) du noyau découpe le temps CPU en tranches (quanta) pour les distribuer aux processus éligibles et simuler une exécution simultanée.
- • PCB (Process Control Block) : Structure de données interne au noyau regroupant les métadonnées de chaque processus (PID, PPID, état, registres CPU, carte mémoire, UID, descripteurs de fichiers).
- • Arborescence et Filiation (PID/PPID) : Tous les processus dérivent d'un arbre généalogique strict démarrant au processus initial systemd ou init (PID 1).
- • Appels système fork() et exec() : fork() clone le processus parent pour engendrer un processus enfant ; exec() remplace le code exécutable du processus enfant par un nouveau programme.
- • Trois États du Processus : Élu (exécute sur le CPU), Prêt (possède ses ressources RAM et attend du temps CPU), Bloqué (suspendu en attente d'une entrée/sortie ou d'un événement externe).

Cas concret enseignant : *L'analogie du livre de cuisine : Le programme stocké sur disque dur est comparé à un livre de recettes sagement rangé dans une bibliothèque (passif, statique, ne consommant ni électricité ni ingrédients). Le processus est le cuisinier en train de préparer activement le plat dans sa cuisine (dynamique, consommant de l'espace en RAM sur le plan de travail, du temps processeur CPU pour la cuisson et utilisant des ustensiles correspondant aux descripteurs de fichiers).*

2. Contrôle et Communication Inter-Processus : Les Signaux POSIX

Les signaux constituent la forme la plus basique et universelle de communication inter-processus (IPC) sous les systèmes d'exploitation POSIX. Lorsqu'un signal est transmis à un processus cible, le noyau interrompt le déroulement séquentiel des instructions du programme pour forcer le traitement du signal. Par défaut, la plupart des signaux entraînent la terminaison du processus cible. Toutefois, un développeur peut capturer un signal et lui

associer une routine personnalisée (signal handler), par exemple pour sauvegarder des fichiers temporaires avant fermeture. Cependant, pour préserver la sécurité du système et permettre à l'administrateur d'éliminer des programmes défaillants ou malveillants, les signaux SIGKILL (9) et SIGSTOP (19) échappent à ce mécanisme : ils sont interceptés et exécutés en direct par le noyau, rendant toute tentative d'interception par le processus impossible.

- Signal POSIX : Interruption logicielle asynchrone transmise par le noyau à un processus pour lui notifier un événement ou lui imposer une instruction.
- Gestionnaire de signal (Signal Handler) : Fonction logicielle enregistrée par le programmeur pour intercepter (catch) un signal au lieu d'exécuter l'action système par défaut.
- SIGINT (Signal 2) : Émis via le raccourci 'Ctrl + C' dans le terminal ; demande l'interruption modérée d'une tâche interactive.
- SIGTERM (Signal 15) : Signal de terminaison propre envoyé par défaut par la commande 'kill'. Il autorise le processus à fermer ses fichiers et sauvegarder son état avant de s'arrêter.
- SIGKILL (Signal 9) : Ordre d'arrêt inconditionnel géré directement par le noyau Linux. Il détruit immédiatement le processus et ne peut être ni intercepté, ni masqué, ni ignoré.
- Job Control (SIGSTOP / SIGCONT) : 'Ctrl + Z' envoie SIGSTOP (Signal 19) pour suspendre l'exécution. Les commandes 'fg' ou 'bg' émettent SIGCONT (Signal 18) pour la reprendre.

Cas concret enseignant : *L'analogie de l'évacuation de l'immeuble : SIGTERM (Signal 15) est le gardien d'immeuble qui frappe poliment à la porte pour vous demander de quitter les lieux (vous disposez du temps nécessaire pour ranger votre bureau, sauvegarder vos travaux et fermer à clé). SIGKILL (Signal 9) représente l'intervention des forces de sécurité qui défoncent la porte et vous expulsent sur-le-champ sans vous laisser la possibilité de toucher à quoi que ce soit.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi la commande 'kill 9 1234' renvoie-t-elle une erreur dans le terminal alors que 'kill -9 1234' fonctionne correctement ?
Réponse du professeur : Le tiret '-' dans 'kill -9 1234' spécifie qu'il s'agit du numéro d'option correspondant au signal à envoyer (Signal 9 / SIGKILL). Sans le tiret, l'outil 'kill' interprète '9' et '1234' comme deux PID distincts et tente d'envoyer le signal par défaut SIGTERM (15) à ces deux identifiants. Si aucun processus n'a le PID 9 ou si vous n'avez pas les droits dessus, la commande échoue avec un message d'erreur.

Question étudiant : Qu'est-ce qu'un processus dit 'Zombie' et comment le détruit-on avec la commande kill ?
Réponse du professeur : Un processus zombie (état Z) est un processus fils qui a terminé son exécution mais dont le parent n'a pas encore lu la valeur de sortie via l'appel système wait(). Le zombie ne consomme plus de processeur ni de mémoire RAM, mais conserve son entrée dans la table des processus du noyau. On ne peut pas éliminer directement un zombie avec 'kill -9' car le processus est déjà mort ! Pour le faire disparaître, il faut soit inciter le processus parent à exécuter wait(), soit tuer le processus parent : le zombie sera alors adopté par le processus racine systemd/init (PID 1), qui le nettoie automatiquement.

■ Exercice & Démonstration d'Application

Énoncé : Dans un terminal Bash Linux, un étudiant saisit la séquence de commandes suivante : 1. sleep 600 & 2. ps -ef | grep sleep 3. kill -15 Explicitez le rôle du symbole '&', détaillez le comportement produit lors de l'envoi du signal -15 et décrivez le résultat obtenu lors d'un contrôle ultérieur par la commande ps.

Résolution pas à pas :
Étape 1 : Le symbole '&' placé à la fin de 'sleep 600 &' indique au terminal d'exécuter la commande en arrière-plan (background). Le shell affiche l'identifiant de job (ex: [1]) ainsi que le PID attribué par le noyau (ex: PID 4521) puis rend immédiatement la main à l'utilisateur. Étape 2 : La commande 'ps -ef | grep sleep' parcourt la liste globale des processus système et filtre le mot-clé 'sleep'. Elle permet d'afficher le PID (4521) ainsi que le PPID (identifiant du terminal Bash hôte). Étape 3 : La commande 'kill -15 4521' transmet le signal SIGTERM (Signal 15) au processus 4521. L'application 'sleep' ne disposant pas d'un gestionnaire personnalisé interceptant ce signal, l'action par défaut du système s'applique : l'arrêt propre et la destruction du processus.

Résultat : Le processus 'sleep 600' est détruit de manière propre. Une seconde vérification au moyen de 'ps -ef | grep sleep' confirme la disparition totale du processus de la table des processus actifs.

■ Auto-Évaluation Flash d'Examen

Q : Quel est le nom et le PID du tout premier processus utilisateur démarré par le noyau Linux au lancement de la machine ?

R : Le premier processus utilisateur est 'systemd' (ou 'init' sur les anciens systèmes Unix) et porte obligatoirement le PID 1.

Q : Quel signal POSIX est généré et transmis au processus en premier plan lorsque l'utilisateur effectue le raccourci 'Ctrl + C' dans le terminal ?

R : La combinaison de touches 'Ctrl + C' envoie le signal SIGINT (Signal 2), demandant l'interruption du processus.

Q : Pourquoi le signal SIGKILL (Signal 9) ne peut-il jamais être intercepté ou ignoré par un programme informatique ?

R : SIGKILL est traité directement et exclusivement par le noyau du système d'exploitation. Le signal n'est jamais transmis au code du programme utilisateur, empêchant tout détournement par un gestionnaire de signal.